

Universal Router SwapProxy and Periphery Audit



Uniswap

March 18, 2026

This document has been prepared solely for the named client and it may not be relied upon by any third parties. The content in this document is provided for informational purposes only and does not constitute professional advice of any kind. Security assessments are limited by factors such as scope, time, and techniques used and therefore this document does not constitute a certification, guarantee, or warranty regarding the proper functioning of any system or the absence of security vulnerabilities.

Copyright © 2026 Zeppelin Group Ltd.

Table of Contents

Table of Contents	2
Summary	3
Scope	4
System Overview	5
Proxy-Based Swap Flow (Universal Router pull request #469)	5
New Payment Behavior (Universal Router pull request #470)	5
Per-Hop Slippage Protections (Universal Router pull request #470)	6
Periphery Slippage and Parameter Updates (v4 Periphery pull request #516)	6
Security Model and Trust Assumptions	6
Medium Severity	8
M-01 Incorrect Bounds Check in toLengthOffset May Disable Per-Hop Slippage Protection	8
M-02 Division by Zero in Exact-Output Swaps Panics on Hook-Subsidized Pools	8
Low Severity	9
L-01 SwapProxy's Execute Function Requires ERC-20 Address for ETH-Only Calls	9
L-02 Solmate SafeTransferLib Silently Succeeds on Non-Contract Token Addresses	9
L-03 Non-Mint Action Streams Allowed in V4_POSITION_MANAGER_CALL	10
L-04 Unnecessary Price Calculations When maxHopSlippage Is Not Set	11
L-05 Incorrect maxPrice Parameter Naming in V4 Periphery Errors	11
L-06 Possible Division-by-Zero Panic When Per-Hop Slippage Is Enabled for Uniswap V2 Swaps	11
L-07 Incorrect Parameter Length Validation in decodeSwapExactInParams and decodeSwapExactOutParams	12
Notes & Additional Information	12
N-01 Unused Tokens and ETH May Get Stuck in Router Contract	12
N-02 Missing Tests for SwapProxy	13
N-03 Fee-on-Transfer Tokens Incur Double Fee Through SwapProxy	13
N-04 Malicious Token Hooks Can Interact with the Router During SwapProxy Execution	14
N-05 Misleading path Length Validation Error	14
N-06 Unnecessary Explicit Cast in V4Router Price Calculation	14
N-07 Duplicated Per-Hop Price Logic and Precision Definitions Across Routers	15
N-08 Misleading maxHopSlippage Name Obscures Per-Hop Price Guard	15
Conclusion	17
Appendix	18
Issue Classification	18

Summary

Type	DeFi	Total Issues	17 (9 resolved)
Timeline	From 2026-02-26 To 2026-03-09	Critical Severity Issues	0 (0 resolved)
Languages	Solidity	High Severity Issues	0 (0 resolved)
		Medium Severity Issues	2 (1 resolved)
		Low Severity Issues	7 (6 resolved)
		Notes & Additional Information	8 (2 resolved)

Scope

OpenZeppelin performed an audit of the following pull requests:

- [pull request #469](#) at commit [7181245](#): Proxy-Based Swap Flow
- [pull request #470](#) at commit [c908caf](#): Universal Router v2.1.1
- [pull request #516](#) at commit [d6d0656](#): Per-hop slippage to single swaps and flip to output/input ratio

In scope were the following files:

```
(pull request #516)
v4-periphery
├── src
│   ├── V4Router.sol
│   ├── interfaces
│   │   └── IV4Router.sol
│   └── libraries
│       └── CalldataDecoder.sol

(pull request #469)
universal-router
├── contracts
│   ├── SwapProxy.sol
│   └── interfaces
│       └── ISwapProxy.sol

(pull request #470)
universal-router
├── contracts
│   ├── base
│   │   └── Dispatcher.sol
│   ├── libraries
│   │   ├── Commands.sol
│   │   └── Constants.sol
│   └── modules
│       ├── Payments.sol
│       └── uniswap
│           ├── v2
│           │   └── V2SwapRouter.sol
│           └── v3
│               ├── BytesLib.sol
│               └── V3SwapRouter.sol
```

System Overview

The Universal Router serves as the unified entry point for executing swaps and other interactions across the Uniswap ecosystem. This audit reviews several recent updates made to the Universal Router that have been designed to enhance routing flexibility, payment precision, and slippage protection. The changes introduce a new proxy-based execution flow, refine payment handling to support fractional amounts, and implement more granular, per-hop slippage controls. Also under review are the updates made to the v4 periphery contracts that expand slippage parameters and modify decoding logic to improve safety against MEV and unexpected price movements.

Proxy-Based Swap Flow (Universal Router pull request #469)

A new [SwapProxy](#) contract facilitates a two-step swap execution path. This allows users to delegate swap execution to a third party without granting direct approval of their tokens to the router.

- **Proxy Execution:** Users approve the [SwapProxy](#) to spend their tokens. The proxy then executes the swap on their behalf through the Universal Router.
- **Context Protection:** The primary goal is to ensure that even when execution is delegated, the user's assets are only used for the intended swap and cannot be otherwise misappropriated within the proxy context.

New Payment Behavior (Universal Router pull request #470)

This update introduces a new payment command capable of handling full-precision amounts. It enables more complex swap scenarios where outputs may be fractional, such as in certain structured products or complex routing paths.

Per-Hop Slippage Protections (Universal Router pull request #470)

To provide more granular control over multi-step swaps, slippage protection can now be specified on a per-hop basis. This is a significant enhancement over global slippage limits, which may not adequately protect against price impact on individual legs of a complex trade.

- **V2/V3 Consistency:** The new slippage model applies uniformly to routes involving both Uniswap V2 and V3 pools.
- **Simple and Multi-Step Swaps:** Protections are validated to work reliably for both single-pool swaps and complex paths that traverse multiple liquidity sources.

Periphery Slippage and Parameter Updates (v4 Periphery pull request #516)

The v4 periphery contracts have been updated to support more expressive slippage configurations and safer parameter handling. These changes aim to mitigate MEV and protect users from unfavorable execution.

- **Expanded Slippage Fields:** Single-swap structures now include additional fields for more precise slippage control.
- **Improved Validation and Decoding:** The logic for validating and decoding swap parameters has been hardened to prevent manipulation and ensure that user-specified constraints are strictly enforced.
- **Precision Adjustments:** Internal precision has been updated to align with the new payment behaviors and slippage models, reducing the risk of regressions or unexpected behavior.

Security Model and Trust Assumptions

The security of the Universal Router and its related components relies on the integrity of the underlying Uniswap protocol contracts, the correctness of off-chain inputs, and the diligence of

the user. The recent updates introduce new dynamics that refine these trust assumptions, particularly concerning delegated execution and parameter validation.

- **User Diligence:** Users are trusted to set appropriate slippage parameters for their swaps. The system provides the tools for granular control, but the responsibility for defining safe execution limits rests with the user.
- **Integrity of Underlying Protocols:** All swaps ultimately execute against Uniswap v2, v3, or v4 pools. The security of these swaps is therefore dependent on the security of the core pool contracts.
- **Correctness of Off-Chain Data:** The construction of swap routes and the calculation of slippage limits are typically performed off-chain. The system assumes that these inputs are generated correctly and are not malicious. The on-chain validation and decoding logic serve as a final safeguard but cannot fully compensate for poorly formed inputs.
- **Double Fee Through the Proxy:** Interacting through [SwapProxy](#) with tokens that contain transfer fees will result in those fees being charged twice. This is due to the transfers from the user to the router and then from the router to the respective pool.

Medium Severity

M-01 Incorrect Bounds Check in `toLengthOffset` May Disable Per-Hop Slippage Protection

The `toLengthOffset` function performs an [incorrect bounds check](#) when decoding dynamic arrays from calldata. The function compares `_bytes.length` with `length + relativeOffset`, where `length` represents the number of array elements rather than the number of bytes.

For arrays of `uint256`, each element occupies `32` bytes. As a result, the check may pass even when the calldata does not contain enough data for all declared elements. When the contract later reads beyond the actual calldata, Solidity returns zero due to calldata zero-padding. If this function is used to decode the `maxHopSlippage` array, missing elements will be interpreted as `0`, which effectively disables slippage protection for those hops because any price ratio greater than or equal to zero is accepted.

A malicious integrator or front-end could construct calldata where the array length is larger than the actual encoded data. Users may believe per-hop slippage protection is enforced while some hops execute without effective checks.

Consider modifying the [bounds check](#) to account for the element size by multiplying `length` by `32` before adding `relativeOffset`.

Update: Resolved at commit [cc89977](#).

M-02 Division by Zero in Exact-Output Swaps Panics on Hook-Subsidized Pools

The `_swapExactOutputSingle` and `_swapExactOutput` functions calculate the price by multiplying `amountOut` by `PRECISION` and dividing the result by `amountIn`, where `amountIn` is the amount returned by the pool after accounting for hook deltas. The issue arises when a hook using `BEFORE_SWAP_RETURNS_DELTA_FLAG` or `AFTER_SWAP_RETURNS_DELTA_FLAG` legitimately reduces the caller's input obligation to zero, for example in subsidized pools. In this case, `amountIn` becomes `0`, which results in a division by zero and triggers an EVM panic.

Consider adding a check for `amountIn` equal to `0` and, in that case, treating the swap as a free swap that satisfies any minimum price threshold.

Update: Acknowledged, not resolved. The team stated:

This behaviour causes a revert for an incredibly niche edgecase, and fixing it would increase gas costs for all swappers. Additionally it is a revert that would be detected on transaction simulation. In our opinion this is not a Medium severity issue.

Low Severity

L-01 SwapProxy's Execute Function Requires ERC-20 Address for ETH-Only Calls

The `SwapProxy`'s `execute` function always requires an `ERC-20 token` address to be provided, even when the user intends to send only ETH via `msg.value`. In scenarios where a user does not want to send any ERC-20 tokens, they are still forced to specify a token address, which can be confusing and unnecessary. It may be more intuitive and user-friendly to check the token amount and call `safeTransferFrom` only when the `amount` is greater than zero.

Consider updating the function logic to allow ETH-only transactions, or if ETH should not be accepted, removing the `payable` modifier.

Update: Resolved at commit [b0162a9](#).

L-02 Solmate SafeTransferLib Silently Succeeds on Non-Contract Token Addresses

The `execute` function in `SwapProxy` uses Solmate's `SafeTransferLib` to perform token transfers, but it does not verify whether the provided token address contains contract code. If a user provides an address without contract code as the token, the `safeTransferFrom` function will succeed silently, and no tokens will actually be transferred. This can lead to unexpected behavior and may mislead users into believing that a transfer has occurred when it has not.

Consider either documenting this behavior or adding an explicit check to ensure that the token address contains contract code.

Update: Resolved at commit [22ee596](#). The [SafeTransferLib](#) behaviour has been documented.

L-03 Non-Mint Action Streams Allowed in `V4_POSITION_MANAGER_CALL`

`UniversalRouter` exposes `V4_POSITION_MANAGER_CALL` in its [command dispatch flow](#), which forwards user-supplied calldata to [modifyLiquidities](#). This is relevant because [modifyLiquidities](#) executes multi-action streams, while position authorization is enforced against the immediate caller context (see [approval check](#), [caller mapping](#), and [lock context](#)). If a position owner has approved `UniversalRouter` as an operator, any external account can invoke the router path that uses that approval context.

The safety gate in `_checkV4PositionManagerCall` only rejects three action bytes (`INCREASE_LIQUIDITY`, `DECREASE_LIQUIDITY`, `BURN_POSITION`) and does not require any mint action. Therefore, non-mint streams pass validation, including `INCREASE_LIQUIDITY_FROM_DELTAS` and payout actions such as `TAKE` / `TAKE_PAIR` (action set: [reference](#); handling: `INCREASE_LIQUIDITY_FROM_DELTAS`, `TAKE`, `TAKE_PAIR`). Since fees are accounted through modify-liquidity delta accounting ([fee accrual path](#), [fee-credit behavior note](#)), an attacker can only supply a minimal delta in `INCREASE_LIQUIDITY_FROM_DELTAS`, realize accrued fees into credit, and transfer those funds to themselves via `TAKE` / `TAKE_PAIR`.

Consider replacing the current blacklist in `_checkV4PositionManagerCall` with a strict allowlist for migration mint flows, and requiring at least one `MINT_POSITION` or `MINT_POSITION_FROM_DELTAS` action in each permitted stream. In addition, consider rejecting any action that can reference existing token IDs or set arbitrary payout recipients, and decoding parameters where necessary to enforce owner-recipient binding. Consider also adding regression coverage that attempts non-mint streams (for example, `INCREASE_LIQUIDITY_FROM_DELTAS` plus `TAKE_PAIR`) through `V4_POSITION_MANAGER_CALL`.

Update: Resolved at commit [74ff047](#). `INCREASE_LIQUIDITY_FROM_DELTAS` has been included in the `_checkV4PositionManagerCall`.

L-04 Unnecessary Price Calculations When `maxHopSlippage` Is Not Set

The `_swapExactInputSingle` and `_swapExactOutputSingle` functions [perform price calculations](#) without first verifying whether `params.maxHopSlippage` is set. Each single-hop swap unconditionally performs multiplication and division operations regardless of whether the `maxHopSlippage` parameter is configured, resulting in unnecessary computational overhead.

Consider adding a conditional check to ensure that `params.maxHopSlippage` is non-zero before performing the associated price calculations and slippage validations. This would avoid redundant computations when slippage protection is not required.

Update: Resolved at commit [0296654](#).

L-05 Incorrect `maxPrice` Parameter Naming in V4 Periphery Errors

After the pricing formula in V4 Periphery changed to division by `amountIn`, the threshold used in per-hop validation now represents a minimum acceptable price, but the corresponding error parameter is still named `maxPrice`. This affects the existing errors `V4TooLittleReceivedPerHop` and `V4TooMuchRequestedPerHop`, as well as the newly added `V4TooLittleReceivedPerHopSingle` and `V4TooMuchRequestedPerHopSingle`. In all four cases, the parameter labeled `maxPrice` actually represents a minimum price threshold, and the NatSpec documentation also incorrectly describes it as a max price. This mismatch can cause confusion for tooling and integrations that decode errors using ABI parameter names since they will display `maxPrice` even though the value represents a minimum threshold.

Consider renaming the `maxPrice` parameter to `minPrice` in all four error definitions and updating the NatSpec comments accordingly.

Update: Resolved at commit [0f447d0](#).

L-06 Possible Division-by-Zero Panic When Per-Hop Slippage Is Enabled for Uniswap V2 Swaps

A [division-by-zero panic](#) error can occur when `amountInput` is equal to zero and per-hop slippage is enabled. This situation arises when a [pair has no extra balance](#) and the

`ALREADY_PAID` signal is zero. In this scenario, the calculation of per-hop slippage attempts to divide by `amountInput`, which results in a panic. There is no risk of fund loss or bypass, but the panic produces a non-descriptive error instead of a meaningful message that indicates the invalid swap condition.

Consider adding a validation check which ensures that `amountInput` is greater than zero before performing per-hop slippage calculations.

Update: Acknowledged, not resolved.

L-07 Incorrect Parameter Length Validation in `decodeSwapExactInParams` and `decodeSwapExactOutParams`

The `decodeSwapExactInParams` and `decodeSwapExactOutParams` functions decode bytes parameters into `ExactInputParams` and `ExactOutputParams` structures. However, these functions do not correctly validate the minimum parameter length because they do not take the `maxHopSlippage` parameter into account. As a result, the validation logic may accept improperly formatted input data.

Consider updating the validation logic in `decodeSwapExactInParams` and `decodeSwapExactOutParams` to ensure the parameter length is correctly checked according to the updated structures, including the `maxHopSlippage` field.

Update: Resolved at commit `d20dca1`.

Notes & Additional Information

N-01 Unused Tokens and ETH May Get Stuck in Router Contract

[Unused tokens and ETH](#) may remain in the Universal Router contract after execution if they are not explicitly swept, as the contract does not automatically return any leftover balances. This can result in unintentionally lost funds when users or integrators fail to include the necessary

sweeping steps in their execution flow. The responsibility for recovering any remaining tokens or ETH lies entirely with the user, who must include sweeping commands to ensure that no funds remain in the contract.

Consider clearly documenting this behavior and emphasizing the need for integrators to incorporate proper sweeping logic to prevent mistakes and potential fund loss.

Update: Acknowledged, not resolved.

N-02 Missing Tests for SwapProxy

The [current implementation of the test suite](#) for [SwapProxy](#) validates the basic happy path for V2 exact input and exact output swaps, as well as simple revert cases for insufficient approval and expired deadlines. However, additional tests could be added:

- Tests for V3 or V4 swap paths through the proxy
- Tests for ETH-only swaps, where `msg.value` is sent with the `amount` set to `0` for the ERC-20 transfer
- Tests for the behavior when a non-contract address is passed as the token parameter, which is not currently covered

Consider implementing the aforementioned tests to improve the test coverage.

Update: Acknowledged, not resolved.

N-03 Fee-on-Transfer Tokens Incur Double Fee Through SwapProxy

The [SwapProxy](#) introduces an extra [token transfer](#) compared to the standard [Permit2](#) flow. With [Permit2](#), tokens move directly from the user's wallet to the destination pool in a single transfer. With [SwapProxy](#), tokens first transfer from the user to the router, then from the router to the pool. For fee-on-transfer tokens, this results in the transfer fee being applied twice, leading to worse execution for the user.

Consider documenting the fact that fee-on-transfer tokens are not currently supported.

Update: Acknowledged, not resolved. The team stated:

We will continue to encourage all integrators to use Permit2 for security reasons, where this does not happen.

N-04 Malicious Token Hooks Can Interact with the Router During SwapProxy Execution

When [transferring tokens](#) from `msg.sender` to the router within the `SwapProxy` contract's `execute` function, the router remains unlocked during the transfer. This creates a window in which a malicious token (or any token that allows an attacker to hijack the execution flow) could execute arbitrary logic and interact with the router while the transfer is in progress.

Since [the router is not locked](#) during this operation, such a token could potentially [call back into the router](#) or trigger unintended interactions before the swap logic proceeds. This behavior does not occur when using `Permit2` signatures, where the token transfer happens while the router is locked, avoiding exposure of the router in the same way.

Consider documenting this behavior, as the current risk is limited to malicious tokens and is therefore considered a user error.

Update: *Acknowledged, not resolved.*

N-05 Misleading `path` Length Validation Error

The `v3SwapExactInput` and `v3SwapExactOutput` functions currently validate the length of the `path` parameter. If the length of `path` is smaller than the size of an address (20 bytes), the functions revert with the `V3InvalidHopSlippageLength` error. The issue is that the error name, `V3InvalidHopSlippageLength`, incorrectly suggests that the problem is related to hop slippage, whereas the actual cause is an invalid path length. This can be misleading for developers and users when debugging or handling errors.

Consider introducing a new error that explicitly indicates an invalid path length and to use this error in the validation logic for `path`. Doing so will improve clarity and correctness in error reporting.

Update: *Resolved at commit [ef061d4](#).*

N-06 Unnecessary Explicit Cast in `V4Router` Price Calculation

`V4Router` computes a per-hop price ratio by multiplying `amountOut` with a `1e36` precision factor and dividing by `amountIn` as part of swap validation. This ratio is compared to user-provided limits to enforce hop-level execution constraints during routing. The explicit

`uint256` cast on `amountOut` in the price expression can be omitted without changing behavior. Because the precision multiplier is already declared as `uint256`, Solidity promotes the arithmetic to `uint256` automatically. As such, removing the cast does not change overflow behavior for this expression under the current type bounds.

Consider removing the explicit cast and standardizing equivalent price expressions in `V4Router` to improve readability while preserving the same runtime behavior and overflow safety.

Update: Acknowledged, not resolved.

N-07 Duplicated Per-Hop Price Logic and Precision Definitions Across Routers

Per-hop slippage validation is implemented in multiple swap modules: `V2SwapRouter`, `V3SwapRouter`, and `V4Router`. In all cases, the router computes a normalized price ratio and compares it to a user-defined threshold to enforce per-hop constraints. The same core logic is maintained in separate code paths, and the precision source is split between `Constants.SLIPPAGE_PRECISION` in universal-router and `V4Router.PRECISION` in v4-periphery. This creates multiple sources of truth for the same scaling and validation behavior, which can lead to divergence over time if updates are not applied consistently across all router implementations.

Consider centralizing per-hop price computation and threshold checks into a shared utility or a common internal pattern, and standardizing precision to one canonical definition across router modules.

Update: Acknowledged, not resolved.

N-08 Misleading `maxHopSlippage` Name Obscures Per-Hop Price Guard

In `V4Router`, `V3SwapRouter`, and `V2SwapRouter`, each hop computes `price = amountOut * 1e36 / amountIn` and compares it against a user-provided threshold. This mechanism is designed to enforce a minimum acceptable exchange rate per hop (output-to-input ratio), and it is used across exact-input and exact-output swap paths. The issue is that the name of the `maxHopSlippage` parameter does not match that behavior.

The current name suggests a slippage cap (often interpreted as bps or percent), while the code expects a minimum price floor in fixed-point [1e36](#) units. This mismatch can cause integrators to configure values with the wrong unit or wrong intuition, which may either weaken protection or trigger unexpected reverts, even though the underlying arithmetic check is implemented consistently.

Consider renaming the parameter to a value-oriented name such as [minHopPriceX36](#) (or equivalent), and updating related structs, errors, and documentation to match the actual semantics. Consider also documenting unit conventions and directionality with explicit numeric examples, and if interface compatibility is required, keeping the current field as a deprecated alias while validating and mapping it internally to the new meaning.

Update: Resolved at commits [e7f31a6](#) and [a262eb8](#).

Conclusion

This audit covered the recent changes made to the Uniswap Universal Router and v4 Periphery contracts, including the new [SwapProxy](#) contract, updated payment logic, and enhanced per-hop slippage controls. Two medium-severity issues were identified: an incorrect bound check when decoding dynamic arrays, which could allow certain hops to execute without effective slippage protection, and a division-by-zero issue for pools whose hooks subsidize swaps. In addition, several low-severity and informational issues were also reported. Overall, aside from the lack of tests for the [SwapProxy](#) contract, the codebase was found to be well-structured and well-tested.

The Uniswap Labs team provided timely context and responses throughout the engagement, which resulted in an effective review.

Appendix

Issue Classification

OpenZeppelin classifies smart contract vulnerabilities on a 5-level scale:

- Critical
- High
- Medium
- Low
- Note/Information

Critical Severity

This classification is applied when the issue's impact is catastrophic, threatening extensive damage to the client's reputation and/or causing severe financial loss to the client or users. The likelihood of exploitation can be high, warranting a swift response. Critical issues typically involve significant risks such as the permanent loss or locking of a large volume of users' sensitive assets or the failure of core system functionalities without viable mitigations. These issues demand immediate attention due to their potential to compromise system integrity or user trust significantly.

High Severity

These issues are characterized by the potential to substantially impact the client's reputation and/or result in considerable financial losses. The likelihood of exploitation is significant, warranting a swift response. Such issues might include temporary loss or locking of a significant number of users' sensitive assets or disruptions to critical system functionalities, albeit with potential, yet limited, mitigations available. The emphasis is on the significant but not always catastrophic effects on system operation or asset security, necessitating prompt and effective remediation.

Medium Severity

Issues classified as being of medium severity can lead to a noticeable negative impact on the client's reputation and/or moderate financial losses. Such issues, if left unattended, have a moderate likelihood of being exploited or may cause unwanted side effects in the system.

These issues are typically confined to a smaller subset of users' sensitive assets or might involve deviations from the specified system design that, while not directly financial in nature, compromise system integrity or user experience. The focus here is on issues that pose a real but contained risk, warranting timely attention to prevent escalation.

Low Severity

Low-severity issues are those that have a low impact on the client's operations and/or reputation. These issues may represent minor risks or inefficiencies to the client's specific business model. They are identified as areas for improvement that, while not urgent, could enhance the security and quality of the codebase if addressed.

Notes & Additional Information Severity

This category is reserved for issues that, despite having a minimal impact, are still important to resolve. Addressing these issues contributes to the overall security posture and code quality improvement but does not require immediate action. It reflects a commitment to maintaining high standards and continuous improvement, even in areas that do not pose immediate risks.